

MapReduce 手推/计算/伪代码预测题

这类题本质就三件事：**算有几个 Map Task / Reduce Task**，**手推 Map 输出和 Shuffle 分组**，**写 Reduce 输出**。PPT 里明确说 MR 作业分为 Map 阶段和 Reduce 阶段，Map 阶段包含 InputFormat、Mapper、Partitioner，Reduce 阶段包含远程拷贝、按 key 排序、Reducer、OutputFormat。

先背总规律：

1. Split 个数 $\approx$ Map Task 个数
2. 默认情况下，Split 通常和 HDFS Block 一一对应
3. Reduce Task 个数由用户设置
4. 每个 Map 会给每个 Reduce 产生一个 partition
5. 中间 key 决定 Shuffle 后怎么分组
6. Partitioner 决定某个 key 去哪个 Reduce

题1：根据文件大小算 Block / Split / Map Task / Reduce Task

题目：HDFS block 大小为 64MB，有一个 300MB 的输入文件。默认 split 与 block 一一对应，用户设置 reduce task 数为 3。问：有几个 HDFS block？几个 split？几个 map task？几个 reduce task？

答案：

```
HDFS block 数 = ceil(300 / 64) = 5  
Split 数 = 5  
Map Task 数 = 5  
Reduce Task 数 = 3
```

详解：300MB 文件按 64MB 切：

```
Block1: 64MB  
Block2: 64MB  
Block3: 64MB  
Block4: 64MB  
Block5: 44MB
```

默认一个 split 对应一个 block，所以有 5 个 split，也就有 5 个 map task。Reduce task 不是由文件大小决定的，而是由用户设置，所以是 3。

考试写法：Map Task 数通常由输入 Split 数决定，默认情况下 Split 和 HDFS Block 一一对应；Reduce Task 数由用户配置决定。

题2：Split 大小和 Block 大小不一样时怎么算

题目： HDFS block 大小为 64MB，输入文件 300MB。如果设置 split size = 128MB，问 block 数、split 数、map task 数分别是多少？

答案：

```
Block 数 = ceil(300 / 64) = 5  
Split 数 = ceil(300 / 128) = 3  
Map Task 数 = 3
```

详解： Block 是 HDFS 的物理存储单位，Split 是 MapReduce 的逻辑计算单位。默认二者常常一一对应，但不是必须一样。

```
Split1: 128MB  
Split2: 128MB  
Split3: 44MB
```

所以 Map Task 数看 Split，不直接看 Block。

考试易错点： 别死记“Map 数 = Block 数”。准确说是：**Map Task 数 = Split 数**；默认 split 与 block 接近一一对应，所以常常等于 block 数。

题3：WordCount 手推 Map 输出、Shuffle 分组、Reduce 输出

题目： 输入两行文本：

```
line1: hello spark  
line2: hello hadoop hello
```

写出 Map 输出、Shuffle 后分组、Reduce 输出。

答案：

Map 输出：

```
line1:  
  emit(hello, 1)  
  emit(spark, 1)  
  
line2:  
  emit(hello, 1)  
  emit(hadoop, 1)  
  emit(hello, 1)
```

Shuffle 分组：

```
hello -> [1, 1, 1]
spark -> [1]
hadoop -> [1]
```

Reduce 输出：

```
hello -> 3
spark -> 1
hadoop -> 1
```

详解： Map 只负责把每个单词拆成 `<word,1>`；Shuffle 自动把相同 key 的 value 聚到一起；Reduce 对 value 列表求和。WordCount 是 MapReduce 最标准的手推题，PPT知识梳理里也把它作为标准答案：Map 输出 `<word,1>`，Reduce 输出 `<word,sum>`。

题4：根据 Partitioner 判断 key 去哪个 Reduce

题目： 有 3 个 Reduce Task, Partitioner 为：

```
partition = hash(key) mod 3
```

已知：

```
hash(apple)=4
hash(banana)=8
hash(cat)=6
hash(dog)=10
```

问每个 key 去哪个 Reduce？

答案：

```
apple:  4 mod 3 = 1  -> Reduce 1
banana: 8 mod 3 = 2  -> Reduce 2
cat:    6 mod 3 = 0  -> Reduce 0
dog:    10 mod 3 = 1 -> Reduce 1
```

详解： Partitioner 不负责聚合，它只决定 **Map 输出的 key-value 发给哪个 Reduce**。相同 key 的 hash 值一样，所以一定会进同一个 Reduce。

考试写法： 默认 Partitioner 通常按 `hash(key) mod R` 分区，R 是 Reduce Task 个数；这样可以保证相同 key 进入同一个 Reduce。

题5：有几个 Map 输出分区？

题目：一个 Job 有 4 个 Map Task，3 个 Reduce Task。问每个 Map 会产生几个 partition？整个 Job 一共有多少个 Map 输出 partition？

答案：

```
每个 Map 产生 3 个 partition  
总 partition 数 = 4 × 3 = 12
```

详解：因为每个 Map 的输出都要按照 Partitioner 分成 R 份，分别给不同的 Reduce。

```
Map1 -> R0, R1, R2  
Map2 -> R0, R1, R2  
Map3 -> R0, R1, R2  
Map4 -> R0, R1, R2
```

Reduce 端会从所有 Map 拉取属于自己的那一份数据。

考试关键词：Map 输出本地中间结果；Reduce 远程拷贝 Map 输出，再排序分组后执行 Reducer。这个流程和 PPT 里 MR 两阶段模型一致。

题6：Combiner 手推题

题目：一个 Map Task 处理如下文本：

```
a b a a b c
```

如果不用 Combiner，Map 输出什么？如果使用 Combiner，Map 端预聚合后输出什么？

不用 Combiner：

```
(a,1)  
(b,1)  
(a,1)  
(a,1)  
(b,1)  
(c,1)
```

使用 Combiner：

```
(a,3)
(b,2)
(c,1)
```

详解：Combiner 可以理解成 Map 端的本地 Reduce，先在一个 Map Task 内部做局部聚合，减少 Shuffle 传输量。

但要注意：Combiner 只适合满足结合律和交换律的操作，比如 **sum**、**count**、**max**、**min**。平均值不能简单把局部平均再平均。

考试写法：Combiner 是可选优化，不改变最终语义，只减少网络传输量。

题7：平均值为什么不能直接用 Combiner 求平均

题目：要求统计每个用户的平均消费。两个 Map Task 处理数据：

```
Map1:
u1 10
u1 20

Map2:
u1 100
```

如果 Map 端直接输出局部平均，会得到什么错误结果？正确写法是什么？

错误做法：

```
Map1 local avg = (10+20)/2 = 15
Map2 local avg = 100

Reduce avg = (15 + 100) / 2 = 57.5
```

真实平均值：

```
(10 + 20 + 100) / 3 = 43.33
```

正确 Map 输出：

```
map(user, value):
    emit(user, (value, 1))
```

正确 Reduce：

```
reduce(user, pairs):  
    sum = 0  
    count = 0  
    for (v, c) in pairs:  
        sum += v  
        count += c  
    emit(user, sum / count)
```

详解： 平均值不能直接“平均局部平均”，因为每个 Map 的样本数量可能不同。正确方式是传 **(sum,count)**，最后统一相除。

考试记法： 求平均：**Map/Combiner** 传 **sum** 和 **count**，不传 **avg**。

题8：倒排索引手推题

题目： 输入文档：

```
doc1: spark hadoop spark  
doc2: hadoop hive  
doc3: spark hive
```

输出每个单词出现在哪些文档中。

Map 输出：

```
doc1:  
    emit(spark, doc1)  
    emit(hadoop, doc1)  
    emit(spark, doc1)  
  
doc2:  
    emit(hadoop, doc2)  
    emit(hive, doc2)  
  
doc3:  
    emit(spark, doc3)  
    emit(hive, doc3)
```

Shuffle 分组：

```
spark  -> [doc1, doc1, doc3]  
hadoop -> [doc1, doc2]  
hive   -> [doc2, doc3]
```

Reduce 输出，去重：

```
spark -> [doc1, doc3]
hadoop -> [doc1, doc2]
hive -> [doc2, doc3]
```

详解：倒排索引的中间 key 是 **word**，因为最后要按单词聚合。Reduce 里最好去重，否则 **spark** 在 doc1 出现两次，会导致 doc1 重复。

考试写法：Map 输出 **<word, doc_id>**；Shuffle 按 word 分组；Reduce 对 doc_id 列表去重并输出。

题9：Map-only 过滤题，Reduce 数量怎么算

题目：有 5 个 split，需要从日志中找出包含 **"ERROR"** 的行。这个任务可以不设置 Reduce。问有几个 Map Task？几个 Reduce Task？输出流程是什么？

答案：

```
Map Task 数 = 5
Reduce Task 数 = 0
```

伪代码：

```
map(offset, line):
    if contains(line, "ERROR"):
        emit(offset, line)
```

详解：如果题目只是过滤，不需要按 key 聚合，就可以写成 map-only job。每个 Map 独立扫描自己的 split，遇到符合条件的行就直接输出。

考试写法：Map-only 任务没有 Shuffle 和 Reduce 阶段，适合过滤、简单转换、格式清洗这类不需要全局聚合的任务。

题10：HBase 输入时 Map Task 数怎么定

题目：使用 MapReduce 扫描 HBase 表做统计。该 HBase 表有 8 个 Region，用户设置 Reduce Task 数为 4。问 Map Task 和 Reduce Task 分别是多少？

答案：

```
Map Task 数 = 8
Reduce Task 数 = 4
```

详解：如果输入来自 HDFS，第一阶段 Map Task 数通常看 split/block；如果输入来自 HBase，MapReduce on HBase 通常每个 Region 对应一个 Map Task。HBase PPT 里也写到，每个 Region 对应一个 Map Task，可以实现分布式并行处理。

考试写法：MapReduce 读取 HBase 时，TableInputFormat 会把表数据转换为输入格式，通常以 Region 为单位并行扫描；因此 Map Task 数由 Region 数决定，Reduce Task 数仍由用户配置决定。

这类题最后怎么做

你考场上按这个顺序写就行：

```
第一步：先算输入 split 数，所以知道 Map Task 数。
第二步：看用户设置的 reducer 数，所以知道 Reduce Task 数。
第三步：Map 输出 <中间key, 中间value>。
第四步：Partitioner 决定 key 去哪个 Reduce。
第五步：Shuffle 按 key 分组。
第六步：Reduce 对每个 key 的 value list 做聚合。
```

最短模板：

```
map(k1, v1):
    提取中间 key
    emit(k2, v2)

shuffle:
    按 k2 分区、排序、分组

reduce(k2, list(v2)):
    聚合 / 去重 / 拼接
    emit(k3, v3)
```

一句话压轴：**Map** 负责拆，**Partitioner** 负责分到哪个 **Reduce**，**Shuffle** 负责按 key 聚，**Reduce** 负责合。